

실전반 6회 · 플랫폼에서 데이터와 온톨로지 발행

- [이번 회차에서 할 일](#)
- [1. 시작 전 준비 \(복습·환경 확인\)](#)
- [2. 핵심 개념](#)
- [3. 활동 1 — Data Fabric에 사전 등록된 교육용 DB 확인 · 50분](#)
- [4. 활동 2 — Ontology Studio에서 제공 클래스 매핑 검토 · 50분](#)
- [5. 활동 3 — 스키마 발행과 설비·최근 상태 질의 · 50분](#)
- [6. 활동 4 — 컨텍스트 조회를 플랫폼 어댑터로 바꾸어 결과 비교 · 50분](#)
- [7. 실습 과제](#)
- [8. 완료 확인 체크리스트](#)
- [9. 회고 \(10분\)](#)
- [10. 실제 플랫폼 대응](#)
- [강사 노트](#)
- [용어](#)

날짜·시간	2027-01-26 (화) 14:00~18:00	모듈	M3 · 원 일정 10번
블록	확장 B4 · B7 / 플랫폼 B3 · B4	산출물	플랫폼 EKG를 조회하는 같은 에이전트
완료 확인	동일 asset_id로 교육용 조회와 플랫폼 조회를 대조		

회차별 블록 로드맵 — 하나의 시스템에 매 회차 기능을 더한다

구현 새로 만든다 · 확장 조건·연결을 바꾼다 · 플랫폼 교육용 플랫폼에 연결한다

실전반 4학년 · 9회 × 4시간 (합수·질의·정책 수정과 검증)

	B1	B2	B3	B4	B5	B6	B7	B8	B9
1회 10/31	구현	구현	구현						
2회 11/07		확장	확장						
3회 11/21			확장	구현					
4회 11/28		확장	확장		구현				구현
5회 01/25				확장		구현	구현		
6회 01/26			플랫폼	확장+P			확장		
7회 01/27						확장+P	확장+P	구현+P	
8회 01/28	확장				확장			확장	
9회 01/29									확장+P

통합반 2·3학년 · 10회 × 4시간 (제공 코드의 매핑·설정·조건 완성)

	B1	B2	B3	B4	B5	B6	B7	B8	B9
1회 11/14	구현	구현							
2회 01/25		확장	구현						
3회 01/26				구현					
4회 01/27		확장	확장		구현				구현
5회 01/28				확장		구현			
6회 01/29							구현	구현	
7회 02/01			플랫폼	확장+P					
8회 02/02						확장+P	확장+P	확장+P	
9회 02/03									확장+P
10회 02/04	확장				확장			확장	확장

이번 회차에서 할 일

1~5회차는 모든 블록을 로컬 코드로 연결했다. 오늘부터 후반부 플랫폼 연결을 시작한다. 수업에서는 실제 uEngine 플랫폼의 개념과 API 모양을 재현한 **교육용 플랫폼(Lab Platform, `http://localhost:8910`)** 을 쓴다. 오늘 다루는 것은 Data Sources 계층의 Data Fabric 과 Ontology 계층의 Ontology Studio 다. 강사가 미리 등록한 교육용 DB(`hydops_edu`)를 확인하고, 제공된 클래스 매핑(설비·센서·SOP·조치는 그래프, 관측·사건·승인·조치 기록·재측정은 원천 DB 가상 연결)을 검토한 뒤 발행해 조회한다. 마지막으로 5회차 에이전트의 `get_asset_context` 도구 본체를 **플랫폼 어댑터**로 바꿔 끼우고, 같은 `asset_id` 에 대해 로컬 조회와 플랫폼 조회가 같은 판단 재료를 주는지 대조한다. 에이전트 코드는 한 줄도 바꾸지 않는다.

학습 목표

- Data Fabric 의 데이터소스 등록 정보·메타데이터·읽기 전용 질의로 교육용 DB 의 테이블과 행 수를 확인한다.
- Ontology Studio 매핑에서 graph 바인딩과 virtual 바인딩을 구분하고, 검증(`validate`)이 잡는 매핑 오류와 발행 전 조회 거부(409)를 재현한다.
- 발행된 스키마에서 객체 조회·관계 탐색·Behavior 호출을 실행하고, "발행"과 "데이터 존재"를 구분해 설명한다.
- `graph.asset_context` 와 같은 모양을 돌려주는 `PlatformContext` 를 고치고, HYD-01·HYD-02 에서 로컬과 플랫폼의 센서·적용 SOP·허용 조치가 같고 속성 차이만 있다는 것을 보인다.

1. 시작 전 준비 (복습·환경 확인)

```
cd lecture/system
curl -s localhost:8910/health; echo
curl -s localhost:8910/studio/schemas
```

강사 준비가 `--upto ontology` 까지면 `published_version` 이 `null` 이고, 강사가 기준 스키마를 발행해 두었으면 아래처럼 1 이다.

```

{"ok":true}
[{"schema_name":"HydraulicOps","description":"유압설비 냉각 이상 대응 - 설비·문서 지식은 그
래프, 수치·실행 이력은 원천 DB 가상 연결 (교육
용)","classes":9,"relationships":8,"published_version":1,"published_at":"2026-09-
14T13:10:04.349585+00:00","dirty":false}]

```

5회차 복습: 에이전트 도구 `get_asset_context(asset_id)` 는 `hydops/b7_agent/tools.py` 의 `get_asset_context_impl` → `graph.asset_context` 를 부른다. 결과는 `asset_id` · `found` · `asset` · `sensors` · `applicable_sops` · `allowed_actions` 여섯 키다. 오늘 이 모양을 플랫폼에서 똑같이 만든다.

2. 핵심 개념

로컬 실습과 플랫폼 실습은 같은 함수를 부른다

전반부는 LangGraph 로 흐름을 직접 짜고, 후반부는 교육용 플랫폼의 프로세스-에이전트-앱이 같은 서비스 함수를 호출한다. 결과는 같은 PostgreSQL 기록으로 대조한다.

로컬 (실전반 1~5회 · 통합반 1~6회)

감지 후 시작	<code>runtime.listeners → Orchestrator.start</code>
근거 확인	<code>LangChain create_agent + SKILL.md</code>
사람 승인	<code>LangGraph interrupt</code> · 관계 화면 승인 버튼
실행·재측정	그래프 노드 → 서비스 함수
체크포인트	<code>PostgresSaver (thread_id = event_id)</code>

공유 서비스 · 도구

```

service.gather_evidence
service.record_decision
service.execute
service.verify
MCP /mcp/ 도구 6종
공유 기록
event · approval ·
action_log · verification

```

교육용 플랫폼 (실전반 6~9회 · 통합반 7~10회)

Watch Agent	SQL 한 건 → rows > 0 → 업무 시작 (인스턴스 키)
업무 에이전트	활당 Skill + MCP 허용 도구 4종
승인 태스크	워크리스트 폼: <code>event_id-asset_id-run_id-제한값-근거</code>
도구 활동	<code>execute_sim_action</code> → <code>verify_recovery</code> 재시도
관계 앱	Ontology 객체(Event) + 프로세스 + 조치 기록이 같은 사건

로컬 실습과 플랫폼 실습은 같은 함수를 부른다

2.1 교육용 플랫폼의 두 모듈

모듈	파일	오늘 쓰는 API
Data Fabric	<code>labplatform/fabric.py</code>	GET <code>/fabric/datasources</code> , POST <code>/fabric/datasources/{name}/query</code> (읽기 전용 트랜잭션, 쓰기 문장 거부)

모듈	파일	오늘 쓰는 API
Ontology Studio	labplatform/studio.py	POST /studio/schemas/import , PUT .../classes/{c}/binding , GET .../validate , POST .../publish , POST .../objects/{c}/fetch , GET .../objects/{c}/{key}/related/{rel} , POST .../objects/{c}/behaviors/{b}/invoke , POST .../golden

2.2 그래프 바인딩과 가상 연결

제공 템플릿 `labplatform/templates/ontology_hydraulic.json` 의 9개 클래스는 두 종류로 나뉜다.

바인딩	클래스	원천	이유
graph (label)	Asset · Sensor · SOP · Action	Neo4j 노드	설비·문서 지식은 관계 탐색이 필요하고 자주 바뀌지 않는다
virtual (datasource·table·column_map)	Observation · Event · Approval · ActionLog · Verification	hydops_edu PostgreSQL 테이블	수치·실행 이력은 원천 DB에 남긴다. 그래프로 복사하지 않는다

관계 8개 중 `HAS_SENSOR · GOVERNED_BY · ALLOWS` 는 그래프 관계, `HAS_EVENT · HAS_OBSERVATION · HAS_APPROVAL · HAS_ACTION · HAS_VERIFICATION` 은 속성 조인이다. `GOVERNED_BY` 는 그래프의 `APPLIES_TO` 를 역방향으로 읽고 `status = active` 만 남긴다.

```
// lecture/system/labplatform/templates/ontology_hydraulic.json (발췌)
{"name": "GOVERNED_BY", "from": "Asset", "to": "SOP", "via": {"kind": "graph", "rel_type": "APPLIES_TO", "direction": "in", "where": {"status": "active"}}},
{"name": "HAS_EVENT", "from": "Asset", "to": "Event", "via": {"kind": "join", "from_property": "asset_id", "to_property": "asset_id"}}
```

주의: Neo4j 에 노드가 있다고 플랫폼 클래스가 되지 않는다. 클래스 매핑과 **발행**이 있어야 조회된다. 반대로 발행됐다고 데이터가 있는 것도 아니다. 발행 ≠ 데이터 존재.

주의: 교육용 플랫폼은 실제 플랫폼이 아니다. 같은 개념을 학생 노트북에서 돌도록 줄인 것이다. 대응 관계는 §10 에 정리한다.

3. 활동 1 — Data Fabric에 사전 등록된 교육용 DB 확인 · 50분

목표

강사가 등록한 `hydops_edu` 데이터소스의 계정·엔진·메타데이터를 확인하고, 읽기 전용 질의로 오늘 조회할 데이터가 실제로 있는지 본다.

따라 하기

1. 콘솔 `http://localhost:8910/console/` → Data Fabric 을 연다.

LP

Lab Platform

교육용 플랫폼 콘솔

개요

DS Data Fabric

ON Ontology Studio

AG Agents

OR Processes

OR Apps

실제 플랫폼을 수업용으로 제한한 환경입니다.

대용 기능은 각 화면 상단에 표시합니다.

Data Fabric

대용: Data Fabric 데이터소스

hydops_edu postgres lab_reader@localhost:55432/hydops PostgreSQL 16.11 on aarch64-unknown-linux-musl

교육용 유압설비 운영 DB (읽기 전용 계정)

run

run_id, source, seed, scenario, started_at, note

approval

approval_id, event_id, approver, decision, action_id, approved_value, comment, decided_at

inspection

id, run_id, lot_id, asset_id, ts, measure, value, isl, usl, unit, is_synthetic

observation

id, run_id, asset_id, sensor_id, ts, elapsed_s, origin_cycle_id, raw_value, value, unit, quality_flag, agg, is_synthetic

event_history

id, event_id, from_status, to_status, actor, detail, at

checkpoint_writes

thread_id, checkpoint_ns, checkpoint_id, task_id, idx, channel, type, blob, task_path

event

event_id, run_id, asset_id, event_type, window_start, window_end, rule_version, evidence, status, status_reason, proposal, process_ref, created_at, updated_at

action_log

action_log_id, event_id, approval_id, action_id, requested_value, idempotency_key, command_status, command_detail, executed_at, sim_ts

checkpoints

thread_id, checkpoint_ns, checkpoint_id, parent_checkpoint_id, type, checkpoint, metadata

verification

verification_id, action_log_id, event_id, window_start, window_end, outcome, metrics, verified_at

checkpoint_blobs

thread_id, checkpoint_ns, channel, version, type, blob

checkpoint_migrations

v

읽기 전용 질의

SELECT event_id, asset_id, status FROM event ORDER BY created_at DESC

실행

쓰기 문장은 거부됩니다.

event_id	asset_id	status
EVT-20260914-130541-HYD-01-61ac	HYD-01	REJECTED
EVT-20260914-130529-HYD-03-3bf1	HYD-03	HOLD_NO_EVIDENCE
EVT-20260914-130444-HYD-01-64c4	HYD-01	ESCALATED
EVT-20260914-130416-HYD-01-07b0	HYD-01	CLOSED
EVT-20260914-130352-HYD-01-2edd	HYD-01	SENSOR_CHECK
EVT-20260914-134527-HYD-01-6294	HYD-01	HOLD_NO_EVIDENCE

Data Fabric 데이터소스와 읽기 전용 질의

2. 같은 내용을 API 로 확인한다.

```
curl -s localhost:8910/fabric/datasources | python3 -c "
import json,sys
for x in json.load(sys.stdin): print(x['name'], x['engine'], x['parameters'],
x['server_version'], x['description'], sorted(x['metadata']['tables']))"
```

3. 설비·센서별 관측 수와 사건 상태 분포를 읽기 전용으로 질의한다.

```
curl -s -XPOST localhost:8910/fabric/datasources/hydops_edu/query -H 'content-type: application/json' \
-d '{"sql":"SELECT asset_id, sensor_id, count(*) AS n FROM observation GROUP BY asset_id, sensor_id ORDER BY asset_id, sensor_id}"'
curl -s -XPOST localhost:8910/fabric/datasources/hydops_edu/query -H 'content-type: application/json' \
-d '{"sql":"SELECT event_type, status, count(*) AS n FROM event GROUP BY event_type, status ORDER BY 1,2}"'
```

4. 쓰기 문장이 어떻게 막히는지 코드로 확인한다(실행하지 않는다).

```
# lecture/system/labplatform/fabric.py (발췌)
if re.search(r"\b(insert|update|delete|drop|alter|create|truncate|grant)\b", sql, re.I):
```

```
raise HTTPException(400, "Data Fabric 질의는 읽기 전용이다")
with psycopg.connect(_dsn(ds["parameters"]), row_factory=dict_row) as c:
    c.execute("SET TRANSACTION READ ONLY")
```

확인 포인트

2단계:

```
hydops_edu postgres {'host': 'localhost', 'port': 55432, 'user': 'lab_reader',
'database': 'hydops', 'password': '*****'} PostgreSQL 16.11 on aarch64-unknown-
linux-musl 교육용 유압설비 운영 DB (읽기 전용 계정) ['action_log', 'approval',
'checkpoint_blobs', 'checkpoint_migrations', 'checkpoint_writes', 'checkpoints',
'event', 'event_history', 'inspection', 'observation', 'run', 'verification']
```

- 계정은 `lab_reader` (SELECT 전용), 비밀번호는 응답에서 가려진다.
- 메타데이터에 LangGraph 체크포인트 테이블(`checkpoints` 등)까지 보인다. 매핑 대상은 그 중 5개(observation · event · approval · action_log · verification)뿐이다.

3단계 (리허설 DB, 수업 DB 는 수가 다르다):

```
{"rows": [{"asset_id": "HYD-01", "sensor_id": "FS1", "n": 24235}, {"asset_id": "HYD-01", "sensor_id": "PS1", "n": 24235}, {"asset_id": "HYD-01", "sensor_id": "TS1", "n": 24235}, {"asset_id": "HYD-02", "sensor_id": "FS1", "n": 23871}, ... , {"asset_id": "HYD-03", "sensor_id": "TS1", "n": 20747}], "count": 9}
{"rows": [{"event_type": "COOLING_ANOMALY", "status": "CLOSED", "n": 2}, {"event_type": "COOLING_ANOMALY", "status": "ESCALATED", "n": 1}, {"event_type": "COOLING_ANOMALY", "status": "HOLD_NO_EVIDENCE", "n": 1}, {"event_type": "COOLING_ANOMALY", "status": "REJECTED", "n": 1}, {"event_type": "PRODUCT_QUALITY", "status": "HOLD_NO_EVIDENCE", "n": 1}, {"event_type": "SENSOR_FAULT", "status": "SENSOR_CHECK", "n": 1}], "count": 6}
```

HYD-02 사건이 **한 건도 없다**는 것을 기록해 둔다. 활동 3 에서 "발행은 되었지만 데이터가 없다"를 만나는 이유다.

흔한 실수

- 데이터소스를 학생이 새로 등록한다. 교육용 DB 는 강사가 스코프된 읽기 전용 계정으로 **사전 등록** 했다. 학생 과제는 확인이다.
- `extract-metadata` 를 다시 눌러 메타데이터를 갱신한다. 공유 문서가 바뀌므로 수업에서는 조회(`GET`)만 한다.

- 질의 결과의 **ts** 를 실제 수집 시각으로 읽는다. 시뮬레이터 재생 시각이다.

4. 활동 2 — Ontology Studio에서 제공 클래스 매핑 검토 · 50분

목표

제공 매핑을 표로 검토하고, 팀 스키마(**Lab_<팀>**)에서 일부러 매핑 오류를 넣어 검증·발행이 어떻게 막는지 본다. 기준 스키마 **HydraulicOps** 는 건드리지 않는다.

따라 하기

1. 콘솔 Ontology Studio 에서 클래스 매핑 표를 연다. 발행 전이면 객체 조회가 거부된다.

LP

Lab Platform

교육용 플랫폼 콘솔

개요

DS Data Fabric

ON **Ontology Studio**

AG Agents

OR Processes

OR Apps

실제 플랫폼을 수업용으로 제한한 환경입니다.

대용 가능한 각 화면 상단에 표시합니다.

Ontology Studio

대용: Ontology Studio 클래스-관계-Behavior 발행

HydraulicOps

미발행

검증

발행

유압설비 냉각 이상 대응 — 설비-문서 지식은 그래프, 수차-실행 이력은 원천 DB 가상 연결 (교육용)

클래스 매핑

클래스	바인딩	원천	속성
Asset Resource 의 수입용 구체 클래스 (유압설비)	그래프	Asset	asset_id , name, cooling, line
Sensor Measure — 온도 압력 유량 센서	그래프	Sensor	sensor_id , code, quantity, unit, hz
SOP 표준 작업 절차 (문서 I D-버전)	그래프	SOP	sop_key , doc_id, version, title, status, event_type
Action 허용 조치와 값 범위	그래프	Action	action_id , name, min_value, max_value, default_value, requires_approval
Observation 관측 (가상 연결 — 원 천 DB 에 남긴다)	가상 연결	hydops_edu.observat ion	id , run_id, asset_id, sensor_id, ts, value, unit, quality_flag
Event 이상 사건 (가상 연결)	가상 연결	hydops_edu.event	event_id , run_id, asset_id, event_type, status, status_reason, rule_version, window_start, created_at
Approval 사람 승인 기록 (가상 연결)	가상 연결	hydops_edu.approval	approval_id , event_id, approver, decision, approved_value, decided_at
ActionLog 조치 실행 기록 — 명령 결과	가상 연결	hydops_edu.action_log	action_log_id , event_id, approval_id, action_id, requested_value, command_status, executed_at
Verification 제측정 판정 — 설비 회 복 결과	가상 연결	hydops_edu.verification	verification_id , event_id, action_log_id, outcome, window_start, window_end

관계

Asset —HAS_SENSOR→ Sensor 그래프

Asset —GOVERNED_BY→ SOP 그래프

SOP —ALLOWS→ Action 그래프

Asset —HAS_EVENT→ Event 조건

Asset —HAS_OBSERVATION→ Observation 조건

Event —HAS_APPROVAL→ Approval 조건

Event —HAS_ACTION→ ActionLog 조건

Event —HAS_VERIFICATION→ Verification 조건

객체 조회

Asset

asset_id

HYD-01

조회

schema HydraulicOps 이 발행되지 않았다 — 조회하려면 먼저 발행한다

Golden Question 검증

HYD-01

event_id

실행

발행 전 객체 조회 거부

2. 기준 스키마 검증 결과를 읽는다(읽기).

```
curl -s localhost:8910/studio/schemas/HydraulicOps/validate
```

3. 팀 스키마를 가져오고, Event의 `status` 속성을 없는 컬럼 `state`에 잘못 매핑한 뒤 검증·발행해 본다. 아래 스크립트의 `N`을 팀 이름으로 바꾼다.

```
PYTHONPATH=. .venv/bin/python - <<'EOF'
import json, httpx
P = "http://localhost:8910"; N = "Lab_teamA"
c = httpx.Client(timeout=30)
t = json.loads(open("labplatform/templates/ontology_hydraulic.json").read());
    t["schema_name"] = N
def show(label, r): print(f"## {label} -> {r.status_code}\n{r.text[:300]}\n")
show("import", c.post(f"{P}/studio/schemas/import", json=t))
show("fetch before publish", c.post(f"{P}/studio/schemas/{N}/objects/Asset/fetch", json={"filters":
    {"asset_id": "HYD-01"}}))
ev = next(x for x in t["classes"] if x["name"] == "Event")
bad = json.loads(json.dumps(ev["binding"])); bad["column_map"][4]["column"] =
    "state"
c.put(f"{P}/studio/schemas/{N}/classes/Event/binding", json=bad)
show("validate bad", c.get(f"{P}/studio/schemas/{N}/validate"))
show("publish bad", c.post(f"{P}/studio/schemas/{N}/publish"))
c.put(f"{P}/studio/schemas/{N}/classes/Event/binding", json=ev["binding"])
show("validate ok", c.get(f"{P}/studio/schemas/{N}/validate"))
EOF
```

확인 포인트

2단계:

```
{"ok":true,"problems":[],"bindings":
[{"class":"Asset","kind":"graph","label":"Asset","nodes":3},
{"class":"Sensor","kind":"graph","label":"Sensor","nodes":9},
{"class":"SOP","kind":"graph","label":"SOP","nodes":7},
{"class":"Action","kind":"graph","label":"Action","nodes":4},
{"class":"Observation","kind":"virtual","table":"observation"},
{"class":"Event","kind":"virtual","table":"event"},
{"class":"Approval","kind":"virtual","table":"approval"},
{"class":"ActionLog","kind":"virtual","table":"action_log"},
{"class":"Verification","kind":"virtual","table":"verification"}]}
```

graph 바인딩은 노드 수(Asset 3 · Sensor 9 · SOP 7 · Action 4)를, virtual 바인딩은 테이블 이름을 보고한다. 검증은 컬럼이 메타데이터에 있는지를 보지 행이 있는지는 보지 않는다.

3단계 (리허설에서 `Lab_P06_Textbook` 으로 실행한 결과, 응답은 앞부분만):

```
## fetch before publish -> 409
{"detail":"schema Lab_P06_Textbook 이 발행되지 않았다 - 조회하려면 먼저 발행한다"}

## validate bad -> 200
{"ok":false,"problems":["Event: 없는 컬럼 ['state']"], ...}

## publish bad -> 422
{"detail":{"error":"검증 실패 - 발행하지 않았다","ok":false,"problems":["Event: 없는 컬럼 ['state']"], ...}}

## validate ok -> 200
{"ok":true,"problems":[], ...}
```

매핑 검토표를 팀별로 채운다.

클래스	키	바인딩	원천	에이전트가 쓰는가
Asset	asset_id	graph	Asset	예 (설비 확인)
Sensor	sensor_id	graph	Sensor	예 (센서 목록)
SOP	sop_key	graph	SOP	예 (적용 SOP)
Action	action_id	graph	Action	예 (허용 조치·값 범위)
Event	event_id	virtual	hydops_edu.event	7회차부터
...				

흔한 실수

- 기준 스키마 `HydraulicOps` 에 오류를 넣어 본다. 다른 팀과 7회차 앱이 같은 스키마를 쓴다. 연습은 `Lab_<팀>` 에서만 한다.
- 검증 `ok: true` 를 "조회하면 데이터가 나온다"로 읽는다. 검증은 구조만 본다.
- SOP 클래스 속성에 `effective_date` · `approver_role` 이 없다는 것을 놓친다. 활동 4 비교에서 차이로 나온다.

5. 활동 3 — 스키마 발행과 설비·최근 상태 질의 · 50분

목표

팀 스키마를 발행하고, 객체 조회·관계 탐색·Behavior 호출·Golden Question 을 실행한다. 로컬 조회와 같은 값이 나오는지 한 가지를 먼저 대조한다.

따라 하기

1. 활동 2 의 팀 스키마를 발행하고 조회한다.

```
PYTHONPATH=. .venv/bin/python - <<'EOF'
import httpx
P = "http://localhost:8910"; N = "Lab_teamA"; c = httpx.Client(timeout=30)
def show(label, r): print(f"## {label} -> {r.status_code}\n{r.text[:400]}\n")
show("publish", c.post(f"{P}/studio/schemas/{N}/publish"))
show("fetch Asset", c.post(f"{P}/studio/schemas/{N}/objects/Asset/fetch", json=
    {"filters": {"asset_id": "HYD-01"}}))
show("fetch Event HYD-03", c.post(f"{P}/studio/schemas/{N}/objects/Event/fetch",
    json={"filters": {"asset_id": "HYD-03"}, "limit": 3}))
show("fetch Event HYD-02", c.post(f"{P}/studio/schemas/{N}/objects/Event/fetch",
    json={"filters": {"asset_id": "HYD-02"}, "limit": 3}))
EOF
```

2. 기준 스키마에서 관계를 따라간다(에이전트 어댑터가 쓸 호출이다).

```
P=http://localhost:8910/studio/schemas/HydraulicOps
curl -s $P/objects/Asset/HYD-02/related/GOVERNED_BY; echo
curl -s "$P/objects/SOP/SOP-C00L-002@v1/related/ALLWS"; echo
```

3. Behavior `Asset.recent_state` 를 부르고, 같은 순간 로컬 도구 `get_recent_window` 와 비교한다.

```
PYTHONPATH=../../labs/practical/S06 .venv/bin/python -c "
import platform_adapter as P
from hydops.b7_agent.tools import get_recent_window_impl
rows = P.PlatformContext().recent_state('HYD-01'); loc =
    get_recent_window_impl('HYD-01', 60)['sensors']
for r in rows:
    l = loc[r['sensor_id']]
```

```
print(r['sensor_id'], 'platform', r['samples'], r['avg_valid'],
      r['max_valid'], r['bad_quality'], '| local', l['samples'],
      l['valid_mean'], l['valid_max'], l['quality']['valid_ratio'],
      l.get('sensor_state'), l.get('sustained_anomaly'))
"
```

4. Golden Question 을 두 설비로 실행한다.

```
curl -s -XPOST localhost:8910/studio/schemas/Lab_teamA/golden -H 'content-type:
application/json' \
-d '{"params":{"asset_id":"HYD-02","event_id":"EVT-NOPE"}}' | python3 -c "
import json,sys
for q in json.load(sys.stdin)['results']: print(q['answered'], q['question'],
      [(s['step'], s.get('count'), s.get('error')) for s in q['steps']])"
```

확인 포인트

1단계:

```
## publish -> 200
{"schema_name":"Lab_P06_Textbook","published_version":1,"classes":9,"relationships":
[...]}
```

```
## fetch Asset -> 200
{"schema_name":"Lab_P06_Textbook","published_version":1,"class":"Asset","binding_kin
[{"asset_id":"HYD-01","name":"유압설비 1호기","cooling":"수랭식 냉각기","line":"L1"}]}
```

```
## fetch Event HYD-03 -> 200
{... "binding_kind":"virtual","count":1,"objects":[{"event_id":"EVT-20260914-
130529-HYD-03-3bf1","run_id":"RUN-ac9c8f2681","asset_id":"HYD-
03","event_type":"COOLING_ANOMALY","status":"HOLD_NO_EVIDENCE","status_reason":"근
거 없음", ...}]}
```

```
## fetch Event HYD-02 -> 200
{... "binding_kind":"virtual","count":0,"objects":[],"note":"발행은 되었지만 조건에 맞는
데이터가 없다 (발행 ≠ 데이터 존재)"}
```

2단계:

```
{"from":"Asset:HYD-
02","relationship":"GOVERNED_BY","to":"SOP","count":4,"objects":[{"sop_key":"SOP-
VER-001@v1",...},{sop_key":"SOP-SEN-001@v1",...},{sop_key":"SOP-ESC-
001@v1",...},{sop_key":"SOP-COOL-002@v1","doc_id":"SOP-COOL-
002","version":1,"title":"냉각 이상 확인 (2호기 공랭
식)","status":"active","event_type":"COOLING_ANOMALY"}]}}
```

```
{
  "from": "SOP:SOP-C00L-002@v1",
  "relationship": "ALL0WS",
  "to": "Action",
  "count": 1,
  "objects": [
    {
      "action_id": "FAN_B00ST",
      "name": "냉각 팬 증속",
      "min_value": 1.0,
      "max_value": 1.5,
      "default_value": 1.3,
      "requires_approval": true
    }
  ]
}
```

관계 탐색 결과의 순서는 로컬 질의(`ORDER BY doc_id`)와 다르다. 어댑터가 정렬해야 하는 이유다.

3단계 (시각에 따라 값이 다르다):

```
FS1 platform 60 7.21 7.41 0 | local 60 7.21 7.41 1.0 None None
PS1 platform 60 120.16 124.79 0 | local 60 120.16 124.79 1.0 None None
TS1 platform 60 54.08 54.72 0 | local 60 54.08 54.72 1.0 VALID False
```

평균·최댓값은 같다. 차이는 **판정 필드**다. Behavior 는 SQL 집계(표본 수·평균·최댓값·품질 불량 수)만 주고, 로컬 도구는 `sensor_state` · `sustained_anomaly` · 최장 결측을 코드로 계산해 준다. 그리고 Behavior SQL 은 `run_id` 조건 없이 설비의 마지막 60초를 본다. 에이전트의 센서 품질 판단(Skill 1 단계)을 오늘 Behavior 로 바꾸지 않는 이유다.

4단계:

```
True HYD-01의 최근 온도는 유효한 센서 관측이며 지속 이상인가? [('설비의 센서 찾기 (그래프)', 3, None), ('최근 60초 상태 (Behavior, 원천 DB)', 3, None)]
True 이 설비에 적용되는 최신 냉각 이상 SOP와 허용 조치는 무엇인가? [('적용 SOP (active 만)', 4, None), ('냉각 이상 SOP 가 허용하는 조치', 1, None)]
False 어떤 근거로 누가 승인했으며 조치 후 새 관측은 회복했는가? [('사건', 0, None), ('승인', None, 'Event EVT-NOPE not found'), ('조치 실행 기록', None, 'Event EVT-NOPE not found'), ('재측정 판정', None, 'Event EVT-NOPE not found')]
```

LP

Lab Platform

교육용 플랫폼 콘솔

개요

DS Data Fabric

ON **Ontology Studio**

AG Agents

OR Processes

OR Apps

실제 플랫폼을 수업용으로 제한한 환경입니다.

대용 가능한 각 화면 상단에 표시됩니다.

Ontology Studio

대용: Ontology Studio 클래스관계 Behavior 발행

HydraulicOps

발행 v1

검증

발행

유압설비 냉각 이상 대응 — 설비-문서 지식은 그래프, 수차-설행 이력은 원천 DB 가상 연결 (교육용)

클래스 매핑

클래스	바인딩	원천	속성
Asset Resource 의 수입품 구체 클래스 (유압설비)	그래프	Asset	asset_id, name, cooling, line
Sensor Measure — 온도-압력 -유량 센서	그래프	Sensor	sensor_id, code, quantity, unit, hz
SOP 표준 작업 절차 (문서 I D-버전)	그래프	SOP	sop_key, doc_id, version, title, status, event_type
Action 허용 조치와 값 범위	그래프	Action	action_id, name, min_value, max_value, default_value, requires_approval
Observation 관측 (가상 연결 — 원 천 DB 에 남긴다)	가상 연결	hydops_edu.observat ion	id, run_id, asset_id, sensor_id, ts, value, unit, quality_flag
Event 이상 사건 (가상 연결)	가상 연결	hydops_edu.event	event_id, run_id, asset_id, event_type, status, status_reason, rule_version, window_start, created_at
Approval 사람 승인 기록 (가상 연 결)	가상 연결	hydops_edu.approval	approval_id, event_id, approver, decision, approved_value, decided_at
ActionLog 조치 실행 기록 — 명령 결과	가상 연결	hydops_edu.action_log	action_log_id, event_id, approval_id, action_id, requested_value, command_status, executed_at
Verification 재측정 판정 — 설비 회 복 결과	가상 연결	hydops_edu.verification	verification_id, event_id, action_log_id, outcome, window_start, window_end

관계

Asset —HAS_SENSOR→ Sensor [그래프](#)

Asset —GOVERNED_BY→ SOP [그래프](#)

SOP —ALLOWS→ Action [그래프](#)

Asset —HAS_EVENT→ Event [초인](#)

Asset —HAS_OBSERVATION→ Observation [초인](#)

Event —HAS_APPROVAL→ Approval [초인](#)

Event —HAS_ACTION→ ActionLog [초인](#)

Event —HAS_VERIFICATION→ Verification [초인](#)

객체 조회

Event

event_id

EVT-20260914-131041-HYD-

조회

v1 - virtual - 1건

event_id	run_id	asset_id	event_type	status	status_reason	rule_version	window
EVT-20260914-131041-HYD-01-f81e	RUN-b5021e42b2	HYD-01-b5021e42b2	COOLING_ANOMALY	CLOSED	재측정 회복 — 종결	temp-sustai n-v1	2026-01-13 10:41:00

Golden Question 검증

HYD-01

EVT-20260914-131041-HYD-01-f81e

실행

✓ HYD-01의 최근 온도는 유효한 센서 관측이며 지속 이상인가?

- 설비의 센서 찾기 (그래프) — 3건 ("sensor_id": "HYD-01-FS1", "code": "FS1", "quantity": "flow", "unit": "L/min", "hz": "10")

- 최근 60초 상태 (Behavior, 원천 DB) — 3건 ("sensor_id": "FS1", "unit": "L/min", "samples": "60", "avg_val": "9.18", "max_val": "9.18", "bad_quality": "0", "last_ts": "2026-09-14T13:3...")

✓ 이 설비에 적용되는 최신 냉각 이상 SOP와 허용 조치는 무엇인가?

- 적용 SOP (active 만) — 5건 ("sop_key": "SOP-VER-001@v1", "doc_id": "SOP-VER-001", "version": "1", "title": "조치 후 재점검", "status": "active", "event_type": "COOLING...")

- 냉각 이상 SOP 가 허용하는 조치 — 1건 ("action_id": "REDUCE_LOAD", "name": "승인 부하 감소", "min_value": "0.6", "max_value": "1", "default_value": "0.8", "requires_approval": true)

✓ 어떤 근거로 누가 승인했으며 조치 후 새 관측은 회복했는가?

- 사건 — 1건 ("event_id": "EVT-20260914-131041-HYD-01-f81e", "run_id": "RUN-b5021e42b2", "asset_id": "HYD-01", "event_type": "COOLING_ANOMALY...")

- 승인 — 1건 ("approval_id": "APR-f5b6f889a6", "event_id": "EVT-20260914-131041-HYD-01-f81e", "approver": "kim.operator", "decision": "APPRO...")

- 조치 실행 기록 — 1건 ("action_log_id": "ACT-d35982d947", "event_id": "EVT-20260914-131041-HYD-01-f81e", "approval_id": "APR-f5b6f889a6", "action_id": "...")

- 재측정 판정 — 1건 ("verification_id": "VER-2ed5682ceb", "event_id": "EVT-20260914-131041-HYD-01-f81e", "action_log_id": "ACT-d35982d947", "outco...

발행 v1, Event 조회, Golden Question 3개 통과

HYD-01 과 실제 CLOSED 사건 ID 로 실행하면 세 질문 모두 **True** (Q2 적용 SOP 5건, Q3 사건·승인·조치·재측정 각 1건)였다.

흔한 실수

- Golden **answered: true** 를 "정답을 맞혔다"로 읽는다. 검증 함수는 각 단계 결과가 **비어 있지 않은지**만 본다. 위 HYD-02 실행에서 Q2 둘째 단계는 템플릿에 고정된 **SOP-C00L-001@v2** (HYD-01 SOP)의 조치를 조회하고도 **True** 였다. 질문 문구도 "HYD-01의..." 로 고정 돼 있다.
- 발행 후 매핑을 바꾸고 다시 발행하지 않는다. 스키마는 **dirty: true** 가 되고 Neo4j 의 **_OntologyModel** 메타는 이전 발행 그대로인데, 교육용 **fetch_objects** 는 저장된 **현재** 매핑으로 조회한다(**studio.py** 는 발행 여부만 확인한다). 응답의 **published_version** 과 실제 매핑이 어긋나므로 바꿨으면 반드시 검증·재발행한다.
- 클래스에 없는 속성으로 거른다. **Asset** 을 **{"educational": true}** 로 **fetch** 하면 **400 {"detail": "Asset 에 속성 educational 가 없다"}** , 정의되지 않은 관계를 따라

가면 `404 {"detail": "relationship NOPE from Asset not defined"}` 가 온다.
로컬 그래프에 있는 속성·관계라도 매핑에 없으면 플랫폼에서는 없다.

6. 활동 4 — 컨텍스트 조회를 플랫폼 어댑터로 바꾸어 결과 비교 · 50 분

목표

`labs/practical/S06/platform_adapter.py` 의 `PlatformContext.asset_context` 와 `compare_contexts` 를 고친다. 고친 어댑터를 에이전트 도구에 끼워 같은 사건의 제안이 로컬과 같은지 확인한다.

따라 하기

1. 시작본 어댑터의 SOP 조회 부분을 읽는다.

```
# lecture/labs/practical/S06/platform_adapter.py (시작본 발췌)
found = self.fetch("Asset", {"asset_id": asset_id}, limit=1)
sensors = self.related("Asset", asset_id, "HAS_SENSOR")["objects"]
sops = self.fetch("SOP", {"status": "active"})["objects"]
allowed = []
for s in sops:
    for a in self.related("SOP", s["sop_key"], "ALLWS")["objects"]:
        allowed.append({k: a.get(k) for k in ACTION_FIELDS})
```

2. 시작본으로 HYD-01·HYD-02 를 대조한다.

```
PYTHONPATH=../../labs/practical/S06 .venv/bin/python -c "
import json, platform_adapter as P
pc = P.PlatformContext()
for a in ['HYD-01', 'HYD-02']:
    p = pc.asset_context(a); l = P.local_context(a)
    print(a, [s['doc_id']+'@v'+str(s['version']) for s in p['applicable_sops']],
          [x['action_id'] for x in p['allowed_actions']])
    print(' ', json.dumps(P.compare_contexts(l, p), ensure_ascii=False))
"
```

3. TODO 를 고친다. SOP 는 `related("Asset", asset_id, "GOVERNED_BY")` 로, 조치는 `action_id` 로 한 번만, 비교는 판단에 쓰는 키의 집합으로, 속성 차이는 `field_gaps` 로

따로 모은다. 같은 명령을 다시 실행하고 HYD-03·HYD-99 도 넣어 본다.

4. 고친 어댑터를 에이전트 도구에 끼운다. `use_context_provider` 는 `tools.get_asset_context_impl` 이름을 잠시 바꿔 치우는 제공 함수다(시스템 파일은 고치지 않는다).

```
PYTHONPATH=../../labs/practical/S06 .venv/bin/python -c "
import platform_adapter as P
from hydops.b7_agent.agent import propose_for_event
pc = P.PlatformContext()
for eid in ['EVT-20260914-130529-HYD-03-3bf1', 'EVT-20260914-131041-HYD-01-f81e',
            'EVT-20260914-130352-HYD-01-2edd']:
    lo = propose_for_event(eid, mode='offline')
    with P.use_context_provider(pc.asset_context):
        pl = propose_for_event(eid, mode='offline')
    print(eid[-11:], lo['proposal']['decision'], pl['proposal']['decision'],
          lo['proposal'] == pl['proposal'], [t['result_summary'] for t in
          pl['tool_trace'] if t['tool'] == 'get_asset_context'])
" 2>&1 |grep -v Warn
```

확인 포인트

2단계, 시작본 (HYD-02 에 HYD-01 의 SOP 와 REDUCE_LOAD 가 섞였다):

```
HYD-01 ['SOP-C00L-001@v2', 'SOP-C00L-002@v1', 'SOP-ESC-001@v1', 'SOP-LOAD-001@v1', 'SOP-SEN-001@v1', 'SOP-VER-001@v1'] ['REDUCE_LOAD', 'FAN_BOOST', 'ESCALATE', 'REDUCE_LOAD', 'SENSOR_CHECK']
{"asset_id": "HYD-01", "found": [true, true], "sensors_equal": true, "sops_equal": false, "actions_equal": false, "only_local": {}, "only_platform": {}, "field_gaps": [], "match": false}
HYD-02 ['SOP-C00L-001@v2', 'SOP-C00L-002@v1', 'SOP-ESC-001@v1', 'SOP-LOAD-001@v1', 'SOP-SEN-001@v1', 'SOP-VER-001@v1'] ['REDUCE_LOAD', 'FAN_BOOST', 'ESCALATE', 'REDUCE_LOAD', 'SENSOR_CHECK']
{"asset_id": "HYD-02", "found": [true, true], "sensors_equal": true, "sops_equal": false, "actions_equal": false, "only_local": {}, "only_platform": {}, "field_gaps": [], "match": false}
```

3단계, 고친 뒤:

asset_id	API 호출 수	match	센서·SOP·조치	field_gaps
HYD-01	8	true	모두 같음	<code>applicable_sops.approver_role</code> , <code>applicable_sops.effective_date</code> , <code>asset.educational</code>
HYD-02	7	true	모두 같음	위와 같음
HYD-03	3	true	SOP 0 · 조치 0 로 같음	<code>asset.educational</code>
HYD-99	1	true	<code>found:</code> <code>[false,</code> <code>false]</code>	—

HYD-02 플랫폼 결과: 적용 SOP `SOP-C00L-002` → `SOP-ESC-001` → `SOP-VER-001` → `SOP-SEN-001`, 허용 조치 `FAN_BOOST(1.0~1.5, 기본 1.3)` · `ESCALATE` · `SENSOR_CHECK`. 로컬 `graph.asset_context('HYD-02')` 와 순서까지 같다. 다른 것은 발행된 SOP 클래스에 없는 `effective_date` · `approver_role` 이 `None` 인 점과, Asset 클래스에 없는 `educational` 속성이다. 딕셔너리 전체 `==` 는 네 설비 중 HYD-99 만 같다고 판정한다. 속성 차이와 판단 재료 불일치를 나눠 보고하는 것이 비교 함수의 목적이다.

4단계:

```
HYD-03-3bf1 HOLD HOLD True [{'sops': [], 'allowed': []}]
HYD-01-f81e ESCALATE ESCALATE True [{'sops': ['SOP-C00L-001 v2', 'SOP-ESC-001 v1', 'SOP-LOAD-001 v1', 'SOP-VER-001 v1', 'SOP-SEN-001 v1'], 'allowed': ['REDUCE_LOAD', 'ESCALATE', 'SENSOR_CHECK']}]
HYD-01-2edd SENSOR_CHECK SENSOR_CHECK True []
```

같은 에이전트 코드가 플랫폼 조회로 같은 제안을 냈다. 센서 오류 사건은 설비 맥락을 조회하지 않으므로 목록이 비어 있다.

같은 4단계를 **시작본 어댑터**로 HYD-03 사건에 돌리면 `get_asset_context` 요약이 `sops` 6건 · `allowed` `REDUCE_LOAD`, `FAN_BOOST`, `ESCALATE`, `REDUCE_LOAD`, `SENSOR_CHECK` 로 오염되지만 결정은 여전히 `HOLD` 였다. `search_sop` 가 로컬 그래프 확인으로 HYD-03 절을 하나도 돌려주지 않았기 때문이다. 한 도구가 틀려도 다른 겹이 막은 것이지 어댑터가 맞은 것이 아니다. HYD-02 사건이었다면 코드상 오프라인 에이전트가 오염된 목록의 첫 설비 조치 `REDUCE_LOAD` 를 고르고,

로컬 그래프를 읽는 `propose_action` 의 `action_allowed_by_sop` 검사에서 실패해 FAN_BOOST 제안 대신 HOLD 가 된다(리허설 DB 에는 HYD-02 사건이 없어 실행으로 확인하지는 않았다).

주의: 오늘 바꾼 것은 `get_asset_context` 한 도구다. `search_sop` 의 그래프 확인과 `propose_action` 의 허용 조치 검사(`executor.allowed_action`)는 여전히 로컬 Neo4j 를 읽는다. "에이전트 전체가 플랫폼으로 옮겨졌다"고 쓰지 않는다.

흔한 실수

- 비교를 딕셔너리 `==` 로 끝낸다. 늘 불일치가 나고, 진짜 불일치(다른 설비 SOP)와 속성 누락을 구분하지 못한다.
- SOP 클래스를 `status` 로만 조회한다. 발행된 클래스는 설비 관계를 모른다. 설비 범위는 관계 `GOVERNED_BY` 가 준다.
- 플랫폼 장애 때 어댑터가 예외를 삼키고 빈 맥락(`found: false`)을 돌려준다. 에이전트가 "없는 설비"로 HOLD 해 원인이 가려진다. 제공 `_json` 은 409-오류를 `PlatformError` 로 올린다.

7. 실습 과제

- 실전반: `labs/practical/S06/` — `platform_adapter.py`
 - `PlatformContext.asset_context` : 없는 설비 · 센서 정렬 · `GOVERNED_BY` 관계와 사건 유형·순서 · 허용 조치 중복 제거
 - `compare_contexts` : 없는 설비 분기 · 판단 키 집합 비교 · `only_local/only_platform` · `field_gaps`
- 검증 방법: 1부는 가짜 플랫폼 응답(`httpx.MockTransport`)으로 어댑터 논리를, 2부는 실행 중인 :8910 의 발행된 `HydraulicOps` 를 읽기 전용(`fetch` · `related` · `invoke`)으로 HYD-01-02-03-99 대조. 어댑터가 GET 과 `fetch` 외의 호출을 하지 않았는지도 본다.
- 확인 명령: `cd lecture/system && PYTHONPATH=. .venv/bin/python -m pytest ../labs/practical/S06 -q`

시작본 실행 결과:

8 failed, 2 passed in 0.71s

모두 고친 뒤: `10 passed` . 플랫폼 서버가 없으면 2부 3개(4개 테스트)가 건너뛰어진다. 그 경우 플랫폼 실습 완료로 기록하지 않는다.

8. 완료 확인 체크리스트

- ☐ `hydops_edu` 가 `lab_reader` 계정의 postgres 데이터소스이고 매핑 대상 5개 테이블이 메타데이터에 있다.
- ☐ 읽기 전용 질의로 설비별 관측 수와 사건 상태 분포를 기록했다.
- ☐ 매핑 검토표에서 graph 4개·virtual 5개 클래스와 키를 채웠다.
- ☐ 팀 스키마에서 발행 전 조회 409, 없는 컬럼 매핑의 검증 실패와 발행 422 를 재현했다.
- ☐ 발행 후 Asset 객체, HYD-03 Event 1건, HYD-02 Event 0건(`발행 ≠ 데이터 존재`)을 확인했다.
- ☐ Behavior `recent_state` 와 로컬 `get_recent_window` 를 같은 순간에 비교하고 판정 필드 차이를 적었다.
- ☐ 같은 asset_id(HYD-01·HYD-02)로 로컬·플랫폼 맥락을 대조해 `match: true` 와 `field_gaps` 를 보고했다.
- ☐ 어댑터를 끼운 에이전트가 세 사건에서 로컬과 같은 제안을 냈다.
- ☐ `pytest ../labs/practical/S06 -q` 가 `10 passed` 다(건너뛴 없이).
- ☐ 팀 `Lab_` 스키마 이름을 강사에게 제출했다.

9. 회고 (10분)

- 발행된 SOP 클래스에 `approver_role` 이 없다. 7회차 승인 태스크에서 "누가 승인하나"를 플랫폼만으로 답해야 한다면 무엇을 바꿔야 하고, 그 변경은 누가(학생·강사·플랫폼 관리자) 해야 하는가?
- Behavior `recent_state` 는 평균·최댓값은 로컬과 같았지만 `sensor_state` 가 없었다. 판정 로직을 SQL Behavior 로 옮기는 것과 도구 코드에 두는 것은 각각 무엇이 좋은가?
- 시작본 어댑터가 HYD-03 에서 틀린 맥락을 줘도 결정은 HOLD 였다. "결과가 같으니 괜찮다"고 판단하면 안 되는 이유를 HYD-02 로 설명한다.

개인 설명 과제 — 내가 설명할 질의 또는 실패 분기 하나: 팀원 각자 다음 중 하나를 골라 2분 안에 설명한다. (a) `GOVERNED_BY` 관계가 `APPLIES_TO` 를 역방향으로 읽어 HYD-02 SOP 4건을 찾는 Cypher(`studio.related`) · (b) 없는 컬럼 매핑이 422 로 막히는 분기 · (c) HYD-02 Event 조회

가 0건인데 오류가 아닌 이유 · (d) `compare_contexts` 의 `field_gaps` 와 `only_platform` 이 다른 문제를 가리키는 이유 · (e) Golden Q2 가 HYD-02 에서 `True` 인데 믿으면 안 되는 이유.

10. 실제 플랫폼 대응

교육용 플랫폼 기능	실제 플랫폼 기능	다른 점
Data Fabric <code>POST /fabric/datasources</code> (engine=postgres), <code>extract-metadata</code> , 읽기 전용 <code>query</code>	Data Fabric 데이터소스	교육용은 PostgreSQL 한 엔진만, 메타데이터를 JSON 문서 한 테이블 (<code>labplatform.doc</code>) 에 둔다. 쓰기 거부는 키워드 검사 + <code>SET TRANSACTION READ ONLY</code> 로 단순화
Ontology Studio 클래스·관계·바인딩(<code>graph / virtual</code>)	Ontology Studio 클래스·관계 매핑	교육용 바인딩은 Neo4j 라벨 또는 테이블 컬럼 대응 두 종류뿐이다. 관계는 그래프 관계형·속성 조인 형 두 종류
<code>GET .../validate</code> , <code>POST .../publish</code> → <code>_OntologyModel/_OntologyClass/_OntologyProperty</code>	Ontology Studio 발행	교육용 검증은 키 속성 1 개·바인딩 존재·컬럼 존재만 본다. 발행 버전은 정수 증가, 이전 버전 조회 기능 없음
<code>objects/{c}/fetch</code> , <code>related/{rel}</code>	발행된 온톨로지 객체 조회·관계 탐색	교육용 필터는 속성 같음 (<code>=</code>) 조건만, 페이지네이션 없음(<code>limit</code>)
Behavior <code>Asset.recent_state</code> (SQL 템플릿)	Ontology Studio Behavior	교육용 Behavior 는 매개변수 SQL 한 문장. 코드 실행·권한 구분 없음
<code>POST .../golden</code>	Golden Question	교육용은 단계 결과가 비어 있지 않은지만 확인한

교육용 플랫폼 기능	실제 플랫폼 기능	다른 점
	검증	다(정답 비교 없음)
학생 <code>PlatformContext</code> 어댑터	에이전트의 온톨로지 조회 연결	교육용은 수업 코드의 도구 본체를 바꿔 끼운다. 실제 플랫폼의 에이전트 연결 방식은 7회차 (MCP·업무 에이전트)에서 다룬다

강사 노트

시간 배분

시각	내용
14:00~14:50	활동 1 Data Fabric (콘솔 둘러보기 15분, API·질의 25분, 사건 분포 기록 10분)
14:50~15:40	활동 2 매핑 검토 (템플릿 읽기 20분, 팀 스키마 오류 재현 30분)
15:40~16:10	휴식 30분
16:10~17:00	활동 3 발행·조회 (팀 스키마 발행, 관계·Behavior·Golden)
17:00~17:50	활동 4 어댑터 수정·대조·에이전트 교체, pytest
17:50~18:00	회고

사전 준비

- 서버: `scripts/servers.sh local 4` (강사). 5회차와 같은 로컬 오케스트레이션이다. 플랫폼 프로세스 모드는 7회차부터.
- `scripts/bootstrap_platform.py --upto ontology` 로 데이터소스 등록·메타데이터·기준 스키마 가져오기까지 한다. 기준 `HydraulicOps` 발행은 수업 시작 전 강사가 **한 번** 한다(`--upto publish`). 학생 발행은 팀별 `Lab_<팀>` 스키마로 한다. 기준 스키마를 여러 팀이 번갈아 가져오거나 발행하면 매핑이 덮이고 7·9회차 앱이 영향을 받는다.
- 활동 4 의 사건 ID 는 수업 DB 에서 HOLD_NO_EVIDENCE · CLOSED · SENSOR_CHECK 하나씩 골라 칠판에 적는다. HYD-02 사건이 있으면 FAN_BOOST 대조에 추가한다.

- 수업 뒤 팀 `Lab_` 스키마 정리: 교육용 플랫폼에는 스키마 삭제 API 가 없다. 강사가 `labplatform.db.delete('ontology', 이름)` 과 해당 `_OntologyModel {name} · _OntologyClass/_OntologyProperty {model}` 노드 삭제를 팀 이름별로 실행한다 (`HydraulicOps` 는 제외). 리허설의 `Lab_P06_Textbook` 은 이 방법으로 지웠다.

장애 대응

- 교육용 플랫폼이 멈추면 활동 1~3 은 S11·S12·S13 스크린샷과 템플릿 JSON 읽기로 진행하고, 활동 4 는 lab 테스트 1부(가짜 응답)로 어댑터 논리를 완성한다. 이 경우 **플랫폼 실습 완료로 기록하지 않고** 7회차 시작 전 30분 보충에서 2부 테스트와 에이전트 교체를 다시 한다.
- `fetch` 가 409 면 기존 스키마가 발행되지 않은 것이다. 학생이 `HydraulicOps` 를 발행하지 않게 하고 강사가 발행한다.
- 메타데이터에 테이블이 없다는 검증 오류가 나면 강사가 `extract-metadata` 를 한 번 실행한다(학생은 실행하지 않는다).

용어

용어	뜻
교육용 플랫폼 (Lab Platform)	실제 uEngine 플랫폼의 개념·API 모양을 수업 환경에 재현한 서버(:8910). 실제 플랫폼이 아니다
데이터소스	Data Fabric 에 등록된 원천 DB 연결 정보. 오늘은 <code>hydops_edu</code> (읽기 전용 계정)
메타데이터 추출	데이터소스의 테이블·컬럼 목록을 읽어 저장하는 것. 가상 연결 검증에 쓴다
클래스 매핑	온톨로지 클래스의 속성을 원천(그래프 라벨 또는 테이블 컬럼)에 대응시키는 정의
graph 바인딩	클래스를 Neo4j 노드 라벨에 연결
virtual 바인딩 (가상 연결)	클래스를 원천 DB 테이블에 연결. 데이터를 복사하지 않고 조회 때 SQL 로 읽는다
발행	검증을 통과한 매핑을 버전으로 고정해 조회 가능하게 하는 것. 데이터 존재와 다르다
Behavior	클래스에 붙인 매개변수 조회(교육용은 SQL 템플릿)
Golden Question	온톨로지가 답해야 하는 업무 질문과 그 조회 단계

용어	뜻
EKG	설비·문서 지식과 원천 데이터를 연결한 기업 지식 그래프. 오늘은 발행된 <code>HydraulicOps</code>
플랫폼 어댑터	에이전트 도구가 기대하는 모양을 유지한 채 조회 원천을 플랫폼 API 로 바꾸는 코드
field_gaps	한쪽(로컬)에만 값이 있는 속성 목록. 판단 재료 불일치와 구분한다